paul graham

on startups

essays, volume five

contents

01	how to get startup ideas		6
02	organic startup ideas		18
03	schlep blindness		22
04	frighteningly ambitious startup ideas		26
05	do things that don't scale		34
06	startup = growth		42
07	before the startup		52
08	startups in 13 sentences		60
09	the reddits	march 2024	64
10	how to start google	march 2024	68
11	founder mode	september 2024	74
12	how universities should prepare founders	august 2026	78
13	how to earn a billion dollars	june 2026	84

november 2012

the way to get startup ideas is not to try to think of startup ideas. it's to look for problems, preferably problems you have yourself.

the very best startup ideas tend to have three things in common: they're something the founders themselves want, that they themselves can build, and that few others realize are worth doing. microsoft, apple, yahoo, google, and facebook all began this way.

problems

why is it so important to work on a problem you have? among other things, it ensures the problem really exists. it sounds obvious to say you should only work on problems that exist. and yet by far the most common mistake startups make is to solve problems no one has.

i made it myself. in 1995 i started a company to put art galleries online. but galleries didn't want to be online. it's not how the art business works. so why did i spend 6 months working on this stupid idea? because i didn't pay attention to users. i invented a model of the world that didn't correspond to reality, and worked from that. i didn't notice my model was wrong until i tried to convince users to pay for what we'd built. even then i took embarrassingly long to catch on. i was attached to my model of the world, and i'd spent a lot of time on the software. they had to want it!

why do so many founders build things no one wants? because they begin by trying to think of startup ideas. that m.o. is doubly dangerous: it doesn't merely yield few good ideas; it yields bad ideas that sound plausible enough to fool you into working on them.

at yc we call these "made-up" or "sitcom" startup ideas. imagine one of the characters on a tv show was starting a startup. the writers would have to invent something for it to do. but coming up with good startup ideas is hard. it's not something you can do for the asking. so (unless they got amazingly lucky) the writers would come up with an idea that sounded plausible, but was actually bad.

for example, a social network for pet owners. it doesn't sound obviously mistaken. millions of people have pets. often they care a lot about their pets and spend a lot of money on them. surely many of these people would like a site where they could talk to other pet owners. not all of them perhaps, but if just 2 or 3 percent were regular visitors, you could have millions of users. you could serve them targeted offers, and maybe charge for premium features. [1]

the danger of an idea like this is that when you run it by your friends with pets, they don't say "i would *never* use this." they say "yeah, maybe i could see using something like that." even when the startup launches, it will sound plausible to a lot of people. they don't want to use it themselves, at least not right now, but they could imagine other people wanting it. sum that reaction across the entire population, and you have zero users. [2]

well

when a startup launches, there have to be at least some users who really need what they're making — not just people who could see themselves using it one day, but who want it urgently. usually this initial group of users is small, for the simple reason that if there were something that large numbers of people urgently needed and that could be built with the amount of effort a startup usually puts into a version one, it would probably already exist. which means you have to compromise on one dimension: you can either build something a large number of people want a small amount, or something a small number of

so if you can't predict whether there's a path out of an idea, how do you choose between ideas? the truth is disappointing but interesting: if you're the right sort of person, you have the right sort of hunches. if you're at the leading edge of a field that's changing fast, when you have a hunch that something is worth doing, you're more likely to be right.

in *zen and the art of motorcycle maintenance*, robert pirsig says:

> you want to know how to paint a perfect painting? it's easy. make yourself perfect and then just paint naturally.

i've wondered about that passage since i read it in high school. i'm not sure how useful his advice is for painting specifically, but it fits this situation well. empirically, the way to have good startup ideas is to become the sort of person who has them.

being at the leading edge of a field doesn't mean you have to be one of the people pushing it forward. you can also be at the leading edge as a user. it was not so much because he was a programmer that facebook seemed a good idea to mark zuckerberg as because he used computers so much. if you'd asked most 40 year olds in 2004 whether they'd like to publish their lives semi-publicly on the internet, they'd have been horrified at the idea. but mark already lived online; to him it seemed natural.

paul buchheit says that people at the leading edge of a rapidly changing field "live in the future." combine that with pirsig and you get:

> live in the future, then build what's missing.

that describes the way many if not most of the biggest startups got started. neither apple nor yahoo nor google nor facebook were even supposed to be companies at first. they grew out of things their founders built because there seemed a gap in the world.

if you look at the way successful founders have had their ideas, it's generally the result of some external stimulus hitting a prepared mind. bill gates and paul allen hear about the altair and think "i bet we could write a basic interpreter for it." drew houston realizes he's forgotten his usb stick and thinks "i really need to make my files live online." lots of people heard about the altair. lots forgot usb sticks. the reason those stimuli caused those founders to start companies was that their experiences had prepared them to notice the opportunities they represented.

the verb you want to be using with respect to startup ideas is not "think up" but "notice." at yc we call ideas that grow naturally out of the founders' own experiences "organic" startup ideas. the most successful startups almost all begin this way.

that may not have been what you wanted to hear. you may have expected recipes for coming up with startup ideas, and instead i'm telling you that the key is to have a mind that's prepared in the right way. but disappointing though it may be, this is the truth. and it is a recipe of a sort, just one that in the worst case takes a year rather than a weekend.

if you're not at the leading edge of some rapidly changing field, you can get to one. for example, anyone reasonably smart can probably get to an edge of programming (e.g. building mobile apps) in a year. since a successful startup will consume at least 3-5 years of your life, a year's preparation would be a reasonable investment. especially if you're also looking for a cofounder. [4]

you don't have to learn programming to be at the leading edge of a domain that's changing fast. other domains change fast. but while learning to hack is not necessary, it is for the foreseeable future sufficient. as marc andreessen put it, software is eating the world, and this trend has decades left to run.

since what you need to do here is loosen up your own mind, it may be best not to make too much of a direct frontal attack on the problem — i.e. to sit down and try to think of ideas. the best plan may be just to keep a background process running, looking for things that seem to be missing. work on hard problems, driven mainly by curiosity, but have a second self watching over your shoulder, taking note of gaps and anomalies. [6]

give yourself some time. you have a lot of control over the rate at which you turn yours into a prepared mind, but you have less control over the stimuli that spark ideas when they hit it. if bill gates and paul allen had constrained themselves to come up with a startup idea in one month, what if they'd chosen a month before the altair appeared? they probably would have worked on a less promising idea. drew houston did work on a less promising idea before dropbox: an sat prep startup. but dropbox was a much better idea, both in the absolute sense and also as a match for his skills. [7]

a good way to trick yourself into noticing ideas is to work on projects that seem like they'd be cool. if you do that, you'll naturally tend to build things that are missing. it wouldn't seem as interesting to build something that already existed.

just as trying to think up startup ideas tends to produce bad ones, working on things that could be dismissed as "toys" often produces good ones. when something is described as a toy, that means it has everything an idea needs except being important. it's cool; users love it; it just doesn't matter. but if you're living in the future and you build something cool that users love, it may matter more than outsiders think. microcomputers seemed like toys when apple and microsoft started working on them. i'm old enough to remember that era; the usual term for people with their own microcomputers was "hobbyists." backrub seemed like an inconsequential science project. the facebook was just a way for undergrads to stalk one another.

at yc we're excited when we meet startups working on things that we could imagine know-it-alls on forums dismissing as toys. to us that's positive evidence an idea is good.

if you can afford to take a long view (and arguably you can't afford not to), you can turn "live in the future and build what's missing" into something even better:

> live in the future and build what seems interesting.

school

that's what i'd advise college students to do, rather than trying to learn about "entrepreneurship." "entrepreneurship" is something you learn best by doing it. the examples of the most successful founders make that clear. what you should be spending your time on in college is ratcheting yourself into the future. college is an incomparable opportunity to do that. what a waste to sacrifice an opportunity to solve the hard part of starting a startup — becoming the sort of person who can have organic startup ideas — by spending time learning about the easy part. especially since you won't even really learn about it, any more than you'd learn about sex in a class. all you'll learn is the words for things.

the clash of domains is a particularly fruitful source of ideas. if you know a lot about programming and you start learning about some other field, you'll probably see problems that software could solve. in fact, you're doubly likely to find good problems in another domain: (a) the inhabitants of that domain are not as likely as software people to have already solved their problems with software, and (b) since you come into the new domain totally ignorant, you don't even know what the status quo is to take it for granted.

have done if they'd followed through on their own insights. google was that type of idea too. the search engines that preceded them shied away from the most radical implications of what they were doing — particularly that the better a job they did, the faster users would leave.

a crowded market is actually a good sign, because it means both that there's demand and that none of the existing solutions are good enough. a startup can't hope to enter a market that's obviously big and yet in which they have no competitors. so any startup that succeeds is either going to be entering a market with existing competitors, but armed with some secret weapon that will get them all the users (like google), or entering a market that looks small but which will turn out to be big (like microsoft). [12]

filters

there are two more filters you'll need to turn off if you want to notice startup ideas: the unsexy filter and the schlep filter.

most programmers wish they could start a startup by just writing some brilliant code, pushing it to a server, and having users pay them lots of money. they'd prefer not to deal with tedious problems or get involved in messy ways with the real world. which is a reasonable preference, because such things slow you down. but this preference is so widespread that the space of convenient startup ideas has been stripped pretty clean. if you let your mind wander a few blocks down the street to the messy, tedious ideas, you'll find valuable ones just sitting there waiting to be implemented.

the schlep filter is so dangerous that i wrote a separate essay about the condition it induces, which i called schlep blindness. i gave stripe as an example of a startup that benefited from turning off this filter, and a pretty striking example it is. thousands of programmers were in a position to see this idea; thousands of programmers knew how painful it was to process payments before stripe. but when they looked for startup ideas they didn't see this one, because unconsciously they shrank from having to deal with payments. and dealing with payments is a schlep for stripe, but not an intolerable one. in fact they might have had net less pain; because the fear of dealing with payments kept most people away from this idea, stripe has had comparatively smooth sailing in other areas that are sometimes painful, like user acquisition. they didn't have to try very hard to make themselves heard by users, because users were desperately waiting for what they were building.

the unsexy filter is similar to the schlep filter, except it keeps you from working on problems you despise rather than ones you fear. we overcame this one to work on viaweb. there were interesting things about the architecture of our software, but we weren't interested in ecommerce per se. we could see the problem was one that needed to be solved though.

turning off the schlep filter is more important than turning off the unsexy filter, because the schlep filter is more likely to be an illusion. and even to the degree it isn't, it's a worse form of self-indulgence. starting a successful startup is going to be fairly laborious no matter what. even if the product doesn't entail a lot of schleps, you'll still have plenty dealing with investors, hiring and firing people, and so on. so if there's some idea you think would be cool but you're kept away from by fear of the schleps involved, don't worry: any sufficiently good idea will have as many.

the unsexy filter, while still a source of error, is not as entirely useless as the schlep filter. if you're at the leading edge of a field that's changing rapidly, your ideas about what's sexy will be somewhat correlated with what's valuable in practice. particularly as you get older and more experienced. plus if you find an idea sexy, you'll work on it more enthusiastically. [13]

recipes

when you find an unmet need that isn't your own, it may be somewhat blurry at first. the person who needs something may not know exactly what they need. in that case i often recommend that founders act like consultants — that they do what they'd do if they'd been retained to solve the problems of this one user. people's problems are similar enough that nearly all the code you write this way will be reusable, and whatever isn't will be a small price to start out certain that you've reached the bottom of the well. [16]

one way to ensure you do a good job solving other people's problems is to make them your own. when rajat suri of e la carte decided to write software for restaurants, he got a job as a waiter to learn how restaurants worked. that may seem like taking things to extremes, but startups are extreme. we love it when founders do such things.

in fact, one strategy i recommend to people who need a new idea is not merely to turn off their schlep and unsexy filters, but to seek out ideas that are unsexy or involve schleps. don't try to start twitter. those ideas are so rare that you can't find them by looking for them. make something unsexy that people will pay you for.

a good trick for bypassing the schlep and to some extent the unsexy filter is to ask what you wish someone else would build, so that you could use it. what would you pay for right now?

since startups often garbage-collect broken companies and industries, it can be a good trick to look for those that are dying, or deserve to, and try to imagine what kind of company would profit from their demise. for example, journalism is in free fall at the moment. but there may still be money to be made from something like journalism. what sort of company might cause people in the future to say "this replaced journalism" on some axis?

but imagine asking that in the future, not now. when one company or industry replaces another, it usually comes in from the side. so don't look for a replacement for x; look for something that people will later say turned out to be a replacement for x. and be imaginative about the axis along which the replacement occurs. traditional journalism, for example, is a way for readers to get information and to kill time, a way for writers to make money and to get attention, and a vehicle for several different types of advertising. it could be replaced on any of these axes (it has already started to be on most).

when startups consume incumbents, they usually start by serving some small but important market that the big players ignore. it's particularly good if there's an admixture of disdain in the big players' attitude, because that often misleads them. for example, after steve wozniak built the computer that became the apple i, he felt obliged to give his then-employer hewlett-packard the option to produce it. fortunately for him, they turned it down, and one of the reasons they did was that it used a tv for a monitor, which seemed intolerably déclassé to a high-end hardware company like hp was at the time. [17]

are there groups of scruffy but sophisticated users like the early microcomputer "hobbyists" that are currently being ignored by the big players? a startup with its sights set on bigger things can often capture a small market easily by expending an effort that wouldn't be justified by that market alone.

similarly, since the most successful startups generally ride some wave bigger than themselves, it could be a good trick to look for waves and ask how one could benefit from them. the prices of gene sequencing and 3d printing are both experiencing moore's law-like declines. what new things will we be able to do in the new world we'll have in a few years? what are we unconsciously ruling out as impossible that will soon be possible?

organic

9 the empirical evidence suggests that if colleges want to help their students start startups, the best thing they can do is leave them alone in the right way.

10 i'm speaking here of it startups; in biotech things are different.

11 this is an instance of a more general rule: focus on users, not competitors. the most important information about competitors is what you learn via users anyway.

12 in practice most successful startups have elements of both. and you can describe each strategy in terms of the other by adjusting the boundaries of what you call the market. but it's useful to consider these two ideas separately.

13 i almost hesitate to raise that point though. startups are businesses; the point of a business is to make money; and with that additional constraint, you can't expect you'll be able to spend all your time working on what interests you most.

14 the need has to be a strong one. you can retroactively describe any made-up idea as something you need. but do you really need that recipe site or local event aggregator as much as drew houston needed dropbox, or brian chesky and joe gebbia needed airbnb?

quite often at yc i find myself asking founders "would you use this thing yourself, if you hadn't written it?" and you'd be surprised how often the answer is no.

15 paul buchheit points out that trying to sell something bad can be a source of better ideas:

"the best technique i've found for dealing with yc companies that have bad ideas is to tell them to go sell the product asap (before wasting time building it). not only do they learn that nobody wants what they are building, they very often come back with a real idea that they discovered in the process of trying to sell the bad idea."

16 here's a recipe that might produce the next facebook, if you're college students. if you have a connection to one of the more powerful sororities at your school, approach the queen bees thereof and offer to be their personal it consultants, building anything they could imagine needing in their social lives that didn't already exist. anything that got built this way would be very promising, because such users are not just the most demanding but also the perfect point to spread from.

i have no idea whether this would work.

17 and the reason it used a tv for a monitor is that steve wozniak started out by solving his own problems. he, like most of his peers, couldn't afford a monitor.

thanks to sam altman, mike arrington, paul buchheit, john collison, patrick collison, garry tan, and harj taggar for reading drafts of this, and marc andreessen, joe gebbia, reid hoffman, shel kaphan, mike moritz and kevin systrom for answering my questions about startup history.

april 2010

the best way to come up with startup ideas is to ask yourself the question: what do you wish someone would make for you?

there are two types of startup ideas: those that grow organically out of your own life, and those that you decide, from afar, are going to be necessary to some class of users other than you. apple was the first type. apple happened because steve wozniak wanted a computer. unlike most people who wanted computers, he could design one, so he did. and since lots of other people wanted the same thing, apple was able to sell enough of them to get the company rolling. they still rely on this principle today, incidentally. the iphone is the phone steve jobs wants. [1]

our own startup, viaweb, was of the second type. we made software for building online stores. we didn't need this software ourselves. we weren't direct marketers. we didn't even know when we started that our users were called "direct marketers." but we were comparatively old when we started the company (i was 30 and robert morris was 29), so we'd seen enough to know users would need this type of software. [2]

there is no sharp line between the two types of ideas, but the most successful startups seem to be closer to the apple type than the viaweb type. when he was writing that first basic interpreter for the altair, bill gates was writing something he would use, as were larry and sergey when they wrote the first versions of google.

organic ideas are generally preferable to the made up kind, but particularly so when the founders are young. it takes experience to predict what other people will want. the worst ideas we see at y combinator are from young founders making things they think other people will want.

so if you want to start a startup and don't know yet what you're going to do, i'd encourage you to focus initially on organic ideas. what's missing or broken in your daily life? sometimes if you just ask that question you'll get immediate answers. it must have seemed obviously broken to bill gates that you could only program the altair in machine language.

you may need to stand outside yourself a bit to see brokenness, because you tend to get used to it and take it for granted. you can be sure it's there, though. there are always great ideas sitting right under our noses. in 2004 it was ridiculous that harvard undergrads were still using a facebook printed on paper. surely that sort of thing should have been online.

there are ideas that obvious lying around now. the reason you're overlooking them is the same reason you'd have overlooked the idea of building facebook in 2004: organic startup ideas usually don't seem like startup ideas at first. we know now that facebook was very successful, but put yourself back in 2004. putting undergraduates' profiles online wouldn't have seemed like much of a startup idea. and in fact, it wasn't initially a startup idea. when mark spoke at a yc dinner this winter he said he wasn't trying to start a company when he wrote the first version of facebook. it was just a project. so was the apple i when woz first started working on it. he didn't think he was starting a company. if these guys had thought they were starting companies, they might have been tempted to do something more "serious," and that would have been a mistake.

so if you want to come up with organic startup ideas, i'd encourage you to focus more on the idea part and less on the startup part. just fix things that seem broken, regardless of whether it seems like the problem is important enough to build a company on. if you keep pursuing such threads it would be

january 2012

there are great startup ideas lying around unexploited right under our noses. one reason we don't see them is a phenomenon i call *schlep blindness*. schlep was originally a yiddish word but has passed into general use in the us. it means a tedious, unpleasant task.

no one likes schleps, but hackers especially dislike them. most hackers who start startups wish they could do it by just writing some clever software, putting it on a server somewhere, and watching the money roll in—without ever having to talk to users, or negotiate with other companies, or deal with other people's broken code. maybe that's possible, but i haven't seen it.

one of the many things we do at y combinator is teach hackers about the inevitability of schleps. no, you can't start a startup by just writing code. i remember going through this realization myself. there was a point in 1995 when i was still trying to convince myself i could start a company by just writing code. but i soon learned from experience that schleps are not merely inevitable, but pretty much what business consists of. a company is defined by the schleps it will undertake. and schleps should be dealt with the same way you'd deal with a cold swimming pool: just jump in. which is not to say you should seek out unpleasant work per se, but that you should never shrink from it if it's on the path to something great.

the most dangerous thing about our dislike of schleps is that much of it is unconscious. your unconscious won't even let you see ideas that involve painful schleps. that's schlep blindness.

the phenomenon isn't limited to startups. most people don't consciously decide not to be in as good physical shape as olympic athletes, for example. their unconscious mind decides for them, shrinking from the work involved.

the most striking example i know of schlep blindness is stripe, or rather stripe's idea. for over a decade, every hacker who'd ever had to process payments online knew how painful the experience was. thousands of people must have known about this problem. and yet when they started startups, they decided to build recipe sites, or aggregators for local events. why? why work on problems few care much about and no one will pay for, when you could fix one of the most important components of the world's infrastructure? because schlep blindness prevented people from even considering the idea of fixing payments.

probably no one who applied to y combinator to work on a recipe site began by asking "should we fix payments, or build a recipe site?" and chose the recipe site. though the idea of fixing payments was right there in plain sight, they never saw it, because their unconscious mind shrank from the complications involved. you'd have to make deals with banks. how do you do that? plus you're moving money, so you're going to have to deal with fraud, and people trying to break into your servers. plus there are probably all sorts of regulations to comply with. it's a lot more intimidating to start a startup like this than a recipe site.

that scariness makes ambitious ideas doubly valuable. in addition to their intrinsic value, they're like undervalued stocks in the sense that there's less demand for them among founders. if you pick an ambitious idea, you'll have less competition, because everyone else will have been frightened off by the challenges involved. (this is also true of starting a startup generally.)

how do you overcome schlep blindness? frankly, the most valuable antidote to schlep blindness is probably ignorance. most successful founders would probably say that if they'd known when they were

march 2012

one of the more surprising things i've noticed while working on y combinator is how frightening the most ambitious startup ideas are. in this essay i'm going to demonstrate this phenomenon by describing some. any one of them could make you a billionaire. that might sound like an attractive prospect, and yet when i describe these ideas you may notice you find yourself shrinking away from them.

don't worry, it's not a sign of weakness. arguably it's a sign of sanity. the biggest startup ideas are terrifying. and not just because they'd be a lot of work. the biggest ideas seem to threaten your identity: you wonder if you'd have enough ambition to carry them through.

there's a scene in *being john malkovich* where the nerdy hero encounters a very attractive, sophisticated woman. she says to him:

> here's the thing: if you ever got me, you wouldn't have a clue what to do with me.

that's what these ideas say to us.

this phenomenon is one of the most important things you can understand about startups. [1] you'd expect big startup ideas to be attractive, but actually they tend to repel you. and that has a bunch of consequences. it means these ideas are invisible to most people who try to think of startup ideas, because their subconscious filters them out. even the most ambitious people are probably best off approaching them obliquely.

1. a new search engine

the best ideas are just on the right side of impossible. i don't know if this one is possible, but there are signs it might be. making a new search engine means competing with google, and recently i've noticed some cracks in their fortress.

the point when it became clear to me that microsoft had lost their way was when they decided to get into the search business. that was not a natural move for microsoft. they did it because they were afraid of google, and google was in the search business. but this meant (a) google was now setting microsoft's agenda, and (b) microsoft's agenda consisted of stuff they weren't good at.

microsoft : google :: google : facebook.

that does not by itself mean there's room for a new search engine, but lately when using google search i've found myself nostalgic for the old days, when google was true to its own slightly aspy self. google used to give me a page of the right answers, fast, with no clutter. now the results seem inspired by the scientologist principle that what's true is what's true for you. and the pages don't have the clean, sparse feel they used to. google search results used to look like the output of a unix utility. now if i accidentally put the cursor in the wrong place, anything might happen.

the way to win here is to build the search engine all the hackers use. a search engine whose users consisted of the top 10,000 hackers and no one else would be in a very powerful position despite its small size, just as google was when it was that search engine. and for the first time in over a decade the idea of switching seems thinkable to me.

since anyone capable of starting this company is one of those 10,000 hackers, the route is at least straightforward: make the search engine you yourself want. feel free to make it excessively hackerish.

different things, and some may look quite different from universities. y combinator itself is arguably one of them.

learning is such a big problem that changing the way people do it will have a wave of secondary effects. for example, the name of the university one went to is treated by a lot of people (correctly or not) as a credential in its own right. if learning breaks up into many little pieces, credentialling may separate from it. there may even need to be replacements for campus social life (and oddly enough, yc even has aspects of that).

you could replace high schools too, but there you face bureaucratic obstacles that would slow down a startup. universities seem the place to start.

4. internet drama

hollywood has been slow to embrace the internet. that was a mistake, because i think we can now call a winner in the race between delivery mechanisms, and it is the internet, not cable.

a lot of the reason is the horribleness of cable clients, also known as tvs. our family didn't wait for apple tv. we hated our last tv so much that a few months ago we replaced it with an imac bolted to the wall. it's a little inconvenient to control it with a wireless mouse, but the overall experience is much better than the nightmare ui we had to deal with before.

some of the attention people currently devote to watching movies and tv can be stolen by things that seem completely unrelated, like social networking apps. more can be stolen by things that are a little more closely related, like games. but there will probably always remain some residual demand for conventional drama, where you sit passively and watch as a plot happens. so how do you deliver drama via the internet? whatever you make will have to be on a larger scale than youtube clips. when people sit down to watch a show, they want to know what they're going to get: either part of a series with familiar characters, or a single longer "movie" whose basic premise they know in advance.

there are two ways delivery and payment could play out. either some company like netflix or apple will be the app store for entertainment, and you'll reach audiences through them. or the would-be app stores will be too overreaching, or too technically inflexible, and companies will arise to supply payment and streaming a la carte to the producers of drama. if that's the way things play out, there will also be a need for such infrastructure companies.

5. the next steve jobs

i was talking recently to someone who knew apple well, and i asked him if the people now running the company would be able to keep creating new things the way apple had under steve jobs. his answer was simply "no." i already feared that would be the answer. i asked more to see how he'd qualify it. but he didn't qualify it at all. no, there will be no more great new stuff beyond whatever's currently in the pipeline. apple's revenues may continue to rise for a long time, but as microsoft shows, revenue is a lagging indicator in the technology business.

so if apple's not going to make the next ipad, who is? none of the existing players. none of them are run by product visionaries, and empirically you can't seem to get those by hiring them. empirically the way you get a product visionary as ceo is for him to found the company and not get fired. so the company that creates the next wave of hardware is probably going to have to be a startup.

i realize it sounds preposterously ambitious for a startup to try to become as big as apple. but no more ambitious than it was for apple to become as big as apple, and they did it. plus a startup taking on this problem now has an advantage the original apple didn't: the example of apple. steve jobs has shown us

i realize how crazy all this sounds. in fact, what i like about this idea is all the different ways in which it's wrong. the whole idea of focusing on optimization is counter to the general trend in software development for the last several decades. trying to write the sufficiently smart compiler is by definition a mistake. and even if it weren't, compilers are the sort of software that's supposed to be created by open source projects, not companies. plus if this works it will deprive all the programmers who take pleasure in making multithreaded apps of so much amusing complexity. the forum troll i have by now internalized doesn't even know where to begin in raising objections to this project. now that's what i call a startup idea.

7. ongoing diagnosis

but wait, here's another that could face even greater resistance: ongoing, automatic medical diagnosis.

one of my tricks for generating startup ideas is to imagine the ways in which we'll seem backward to future generations. and i'm pretty sure that to people 50 or 100 years in the future, it will seem barbaric that people in our era waited till they had symptoms to be diagnosed with conditions like heart disease and cancer.

for example, in 2004 bill clinton found he was feeling short of breath. doctors discovered that several of his arteries were over 90% blocked and 3 days later he had a quadruple bypass. it seems reasonable to assume bill clinton has the best medical care available. and yet even he had to wait till his arteries were over 90% blocked to learn that the number was over 90%. surely at some point in the future we'll know these numbers the way we now know something like our weight. ditto for cancer. it will seem preposterous to future generations that we wait till patients have physical symptoms to be diagnosed with cancer. cancer will show up on some sort of radar screen immediately.

(of course, what shows up on the radar screen may be different from what we think of now as cancer. i wouldn't be surprised if at any given time we have ten or even hundreds of microcancers going at once, none of which normally amount to anything.)

a lot of the obstacles to ongoing diagnosis will come from the fact that it's going against the grain of the medical profession. the way medicine has always worked is that patients come to doctors with problems, and the doctors figure out what's wrong. a lot of doctors don't like the idea of going on the medical equivalent of what lawyers call a "fishing expedition," where you go looking for problems without knowing what you're looking for. they call the things that get discovered this way "incidentalomas," and they are something of a nuisance.

for example, a friend of mine once had her brain scanned as part of a study. she was horrified when the doctors running the study discovered what appeared to be a large tumor. after further testing, it turned out to be a harmless cyst. but it cost her a few days of terror. a lot of doctors worry that if you start scanning people with no symptoms, you'll get this on a giant scale: a huge number of false alarms that make patients panic and require expensive and perhaps even dangerous tests to resolve. but i think that's just an artifact of current limitations. if people were scanned all the time and we got better at deciding what was a real problem, my friend would have known about this cyst her whole life and known it was harmless, just as we do a birthmark.

there is room for a lot of startups here. in addition to the technical obstacles all startups face, and the bureaucratic obstacles all medical startups face, they'll be going against thousands of years of medical tradition. but it will happen, and it will be a great thing—so great that people in the future will feel as sorry for us as we do for the generations that lived before anaesthesia and antibiotics.

tactics

july 2013

one of the most common types of advice we give at y combinator is to do things that don't scale. a lot of would-be founders believe that startups either take off or don't. you build something, make it available, and if you've made a better mousetrap, people beat a path to your door as promised. or they don't, in which case the market must not exist. [1]

actually startups take off because the founders make them take off. there may be a handful that just grew by themselves, but usually it takes some sort of push to get them going. a good metaphor would be the cranks that car engines had before they got electric starters. once the engine was going, it would keep going, but there was a separate and laborious process to get it going.

recruit

the most common unscalable thing founders have to do at the start is to recruit users manually. nearly all startups have to. you can't wait for users to come to you. you have to go out and get them.

stripe is one of the most successful startups we've funded, and the problem they solved was an urgent one. if anyone could have sat back and waited for users, it was stripe. but in fact they're famous within yc for aggressive early user acquisition.

startups building things for other startups have a big pool of potential users in the other companies we've funded, and none took better advantage of it than stripe. at yc we use the term "collison installation" for the technique they invented. more diffident founders ask "will you try our beta?" and if the answer is yes, they say "great, we'll send you a link." but the collison brothers weren't going to wait. when anyone agreed to try stripe they'd say "right then, give me your laptop" and set them up on the spot.

there are two reasons founders resist going out and recruiting users individually. one is a combination of shyness and laziness. they'd rather sit at home writing code than go out and talk to a bunch of strangers and probably be rejected by most of them. but for a startup to succeed, at least one founder (usually the ceo) will have to spend a lot of time on sales and marketing. [2]

the other reason founders ignore this path is that the absolute numbers seem so small at first. this can't be how the big, famous startups got started, they think. the mistake they make is to underestimate the power of compound growth. we encourage every startup to measure their progress by weekly growth rate. if you have 100 users, you need to get 10 more next week to grow 10% a week. and while 110 may not seem much better than 100, if you keep growing at 10% a week you'll be surprised how big the numbers get. after a year you'll have 14,000 users, and after 2 years you'll have 2 million.

you'll be doing different things when you're acquiring users a thousand at a time, and growth has to slow down eventually. but if the market exists you can usually start by recruiting users manually and then gradually switch to less manual methods. [3]

airbnb is a classic example of this technique. marketplaces are so hard to get rolling that you should expect to take heroic measures at first. in airbnb's case, these consisted of going door to door in new york, recruiting new users and helping existing ones improve their listings. when i remember the airbnbs during yc, i picture them with rolly bags, because when they showed up for tuesday dinners they'd always just flown back from somewhere.

fragile

have too many to do so much for. that would be a great problem to have. see if you can make it happen. and incidentally, when it does, you'll find that delighting customers scales better than you expected. partly because you can usually find ways to make anything scale more than you would have predicted, and partly because delighting customers will by then have permeated your culture.

i have never once seen a startup lured down a blind alley by trying too hard to make their initial users happy.

but perhaps the biggest thing preventing founders from realizing how attentive they could be to their users is that they've never experienced such attention themselves. their standards for customer service have been set by the companies they've been customers of, which are mostly big ones. tim cook doesn't send you a hand-written note after you buy a laptop. he can't. but you can. that's one advantage of being small: you can provide a level of service no big company can. [6]

once you realize that existing conventions are not the upper bound on user experience, it's interesting in a very pleasant way to think about how far you could go to delight your users.

experience

i was trying to think of a phrase to convey how extreme your attention to users should be, and i realized steve jobs had already done it: insanely great. steve wasn't just using "insanely" as a synonym for "very." he meant it more literally — that one should focus on quality of execution to a degree that in everyday life would be considered pathological.

all the most successful startups we've funded have, and that probably doesn't surprise would-be founders. what novice founders don't get is what insanely great translates to in a larval startup. when steve jobs started using that phrase, apple was already an established company. he meant the mac (and its documentation and even packaging — such is the nature of obsession) should be insanely well designed and manufactured. that's not hard for engineers to grasp. it's just a more extreme version of designing a robust and elegant product.

what founders have a hard time grasping (and steve himself might have had a hard time grasping) is what insanely great morphs into as you roll the time slider back to the first couple months of a startup's life. it's not the product that should be insanely great, but the experience of being your user. the product is just one component of that. for a big company it's necessarily the dominant one. but you can and should give users an insanely great experience with an early, incomplete, buggy product, if you make up the difference with attentiveness.

can, perhaps, but should? yes. over-engaging with early users is not just a permissible technique for getting growth rolling. for most successful startups it's a necessary part of the feedback loop that makes the product good. making a better mousetrap is not an atomic operation. even if you start the way most successful startups have, by building something you yourself need, the first thing you build is never quite right. and except in domains with big penalties for making mistakes, it's often better not to aim for perfection initially. in software, especially, it usually works best to get something in front of users as soon as it has a quantum of utility, and then see what they do with it. perfectionism is often an excuse for procrastination, and in any case your initial model of users is always inaccurate, even if you're one of them. [7]

the feedback you get from engaging directly with your earliest users will be the best you ever get. when you're so big you have to resort to focus groups, you'll wish you could go over to your users' homes and offices and watch them use your stuff like you did when there were only a handful of them.

sometimes we advise founders of b2b startups to take over-engagement to an extreme, and to pick a single user and act as if they were consultants building something just for that one user. the initial user serves as the form for your mold; keep tweaking till you fit their needs perfectly, and you'll usually find you've made something other users want too. even if there aren't many of them, there are probably adjacent territories that have more. as long as you can find just one user who really needs something and can act on that need, you've got a toehold in making something people want, and that's as much as any startup needs initially. [9]

consulting is the canonical example of work that doesn't scale. but (like other ways of bestowing one's favors liberally) it's safe to do it so long as you're not being paid to. that's where companies cross the line. so long as you're a product company that's merely being extra attentive to a customer, they're very grateful even if you don't solve all their problems. but when they start paying you specifically for that attentiveness — when they start paying you by the hour — they expect you to do everything.

another consulting-like technique for recruiting initially lukewarm users is to use your software yourselves on their behalf. we did that at viaweb. when we approached merchants asking if they wanted to use our software to make online stores, some said no, but they'd let us make one for them. since we would do anything to get users, we did. we felt pretty lame at the time. instead of organizing big strategic e-commerce partnerships, we were trying to sell luggage and pens and men's shirts. but in retrospect it was exactly the right thing to do, because it taught us how it would feel to merchants to use our software. sometimes the feedback loop was near instantaneous: in the middle of building some merchant's site i'd find i needed a feature we didn't have, so i'd spend a couple hours implementing it and then resume building the site.

manual

there's a more extreme variant where you don't just use your software, but are your software. when you only have a small number of users, you can sometimes get away with doing by hand things that you plan to automate later. this lets you launch faster, and when you do finally automate yourself out of the loop, you'll know exactly what to build because you'll have muscle memory from doing it yourself.

when manual components look to the user like software, this technique starts to have aspects of a practical joke. for example, the way stripe delivered "instant" merchant accounts to its first users was that the founders manually signed them up for traditional merchant accounts behind the scenes.

some startups could be entirely manual at first. if you can find someone with a problem that needs solving and you can solve it manually, go ahead and do that for as long as you can, and then gradually automate the bottlenecks. it would be a little frightening to be solving users' problems in a way that wasn't yet automatic, but less frightening than the far more common case of having something automatic that doesn't yet solve anyone's problems.

big

i should mention one sort of initial tactic that usually doesn't work: the big launch. i occasionally meet founders who seem to believe startups are projectiles rather than powered aircraft, and that they'll make it big if and only if they're launched with sufficient initial velocity. they want to launch simultaneously in 8 different publications, with embargoes. and on a tuesday, of course, since they read somewhere that's the optimum day to launch something.

it's easy to see how little launches matter. think of some successful startups. how many of their launches do you remember? all you need from a launch is some initial core of users. how well you're doing a

3 the reason this works is that as you get bigger, your size helps you grow. patrick collison wrote "at some point, there was a very noticeable change in how stripe felt. it tipped from being this boulder we had to push to being a train car that in fact had its own momentum."

4 one of the more subtle ways in which yc can help founders is by calibrating their ambitions, because we know exactly how a lot of successful startups looked when they were just getting started.

5 if you're building something for which you can't easily get a small set of users to observe — e.g. enterprise software — and in a domain where you have no connections, you'll have to rely on cold calls and introductions. but should you even be working on such an idea?

6 garry tan pointed out an interesting trap founders fall into in the beginning. they want so much to seem big that they imitate even the flaws of big companies, like indifference to individual users. this seems to them more "professional." actually it's better to embrace the fact that you're small and use whatever advantages that brings.

7 your user model almost couldn't be perfectly accurate, because users' needs often change in response to what you build for them. build them a microcomputer, and suddenly they need to run spreadsheets on it, because the arrival of your new microcomputer causes someone to invent the spreadsheet.

8 if you have to choose between the subset that will sign up quickest and those that will pay the most, it's usually best to pick the former, because those are probably the early adopters. they'll have a better influence on your product, and they won't make you expend as much effort on sales. and though they have less money, you don't need that much to maintain your target growth rate early on.

9 yes, i can imagine cases where you could end up making something that was really only useful for one user. but those are usually obvious, even to inexperienced founders. so if it's not obvious you'd be making something for a market of one, don't worry about that danger.

10 there may even be an inverse correlation between launch magnitude and success. the only launches i remember are famous flops like the segway and google wave. wave is a particularly alarming example, because i think it was actually a great idea that was killed partly by its overdone launch.

11 google grew big on the back of yahoo, but that wasn't a partnership. yahoo was their customer.

12 it will also remind founders that an idea where the second component is empty — an idea where there is nothing you can do to get going, e.g. because you have no way to find users to recruit manually — is probably a bad idea, at least for those founders.

thanks to sam altman, paul buchheit, patrick collison, kevin hale, steven levy, jessica livingston, geoff ralston, and garry tan for reading drafts of this.

september 2012

a startup is a company designed to grow fast. being newly founded does not in itself make a company a startup. nor is it necessary for a startup to work on technology, or take venture funding, or have some sort of "exit." the only essential thing is growth. everything else we associate with startups follows from growth.

if you want to start one it's important to understand that. startups are so hard that you can't be pointed off to the side and hope to succeed. you have to know that growth is what you're after. the good news is, if you get growth, everything else tends to fall into place. which means you can use growth like a compass to make almost every decision you face.

redwoods

let's start with a distinction that should be obvious but is often overlooked: not every newly founded company is a startup. millions of companies are started every year in the us. only a tiny fraction are startups. most are service businesses — restaurants, barbershops, plumbers, and so on. these are not startups, except in a few unusual cases. a barbershop isn't designed to grow fast. whereas a search engine, for example, is.

when i say startups are designed to grow fast, i mean it in two senses. partly i mean designed in the sense of intended, because most startups fail. but i also mean startups are different by nature, in the same way a redwood seedling has a different destiny from a bean sprout.

that difference is why there's a distinct word, "startup," for companies designed to grow fast. if all companies were essentially similar, but some through luck or the efforts of their founders ended up growing very fast, we wouldn't need a separate word. we could just talk about super-successful companies and less successful ones. but in fact startups do have a different sort of dna from other businesses. google is not just a barbershop whose founders were unusually lucky and hard-working. google was different from the beginning.

to grow rapidly, you need to make something you can sell to a big market. that's the difference between google and a barbershop. a barbershop doesn't scale.

for a company to grow really big, it must (a) make something lots of people want, and (b) reach and serve all those people. barbershops are doing fine in the (a) department. almost everyone needs their hair cut. the problem for a barbershop, as for any retail establishment, is (b). a barbershop serves customers in person, and few will travel far for a haircut. and even if they did, the barbershop couldn't accommodate them. [1]

writing software is a great way to solve (b), but you can still end up constrained in (a). if you write software to teach tibetan to hungarian speakers, you'll be able to reach most of the people who want it, but there won't be many of them. if you make software to teach english to chinese speakers, however, you're in startup territory.

most businesses are tightly constrained in (a) or (b). the distinctive feature of successful startups is that they're not.

ideas

it might seem that it would always be better to start a startup than an ordinary business. if you're going to start a company, why not start the type with the most potential? the catch is that this is a (fairly) eff-

both begins with an idea exposed by technological change and makes a product consisting of technology in the narrower sense (what used to be called "high technology"), it's easy to conflate the two. but the two connections are distinct and in principle one could start a startup that was neither driven by technological change, nor whose product consisted of technology except in the broader sense. [4]

rate

how fast does a company have to grow to be considered a startup? there's no precise answer to that. "startup" is a pole, not a threshold. starting one is at first no more than a declaration of one's ambitions. you're committing not just to starting a company, but to starting a fast growing one, and you're thus committing to search for one of the rare ideas of that type. but at first you have no more than commitment. starting a startup is like being an actor in that respect. "actor" too is a pole rather than a threshold. at the beginning of his career, an actor is a waiter who goes to auditions. getting work makes him a successful actor, but he doesn't only become an actor when he's successful.

so the real question is not what growth rate makes a company a startup, but what growth rate successful startups tend to have. for founders that's more than a theoretical question, because it's equivalent to asking if they're on the right path.

the growth of a successful startup usually has three phases:

1. there's an initial period of slow or no growth while the startup tries to figure out what it's doing.
2. as the startup figures out how to make something lots of people want and how to reach those people, there's a period of rapid growth.
3. eventually a successful startup will grow into a big company. growth will slow, partly due to internal limits and partly because the company is starting to bump up against the limits of the markets it serves. [5]

together these three phases produce an s-curve. the phase whose growth defines the startup is the second one, the ascent. its length and slope determine how big the company will be.

the slope is the company's growth rate. if there's one number every founder should always know, it's the company's growth rate. that's the measure of a startup. if you don't know that number, you don't even know if you're doing well or badly.

when i first meet founders and ask what their growth rate is, sometimes they tell me "we get about a hundred new customers a month." that's not a rate. what matters is not the absolute number of new customers, but the ratio of new customers to existing ones. if you're really getting a constant number of new customers every month, you're in trouble, because that means your growth rate is decreasing.

during y combinator we measure growth rate per week, partly because there is so little time before demo day, and partly because startups early on need frequent feedback from their users to tweak what they're doing. [6]

a good growth rate during yc is 5-7% a week. if you can hit 10% a week you're doing exceptionally well. if you can only manage 1%, it's a sign you haven't yet figured out what you're doing.

the best thing to measure the growth rate of is revenue. the next best, for startups that aren't charging initially, is active users. that's a reasonable proxy for revenue growth because whenever the startup does start trying to make money, their revenues will probably be a constant multiple of active users. [7]

compass

you'll discover cooler things than you could ever have made up. for startups, growth is a constraint much like truth. every successful startup is at least partly a product of the imagination of growth. [9]

value

it's hard to find something that grows consistently at several percent a week, but if you do you may have found something surprisingly valuable. if we project forward we see why.

weeklyyearly 1%1.7x 2%2.8x 5%12.6x 7%33.7x 10%142.0x

a company that grows at 1% a week will grow 1.7x a year, whereas a company that grows at 5% a week will grow 12.6x. a company making $1000 a month (a typical number early in yc) and growing at 1% a week will 4 years later be making $7900 a month, which is less than a good programmer makes in salary in silicon valley. a startup that grows at 5% a week will in 4 years be making $25 million a month. [10]

our ancestors must rarely have encountered cases of exponential growth, because our intuitions are no guide here. what happens to fast growing startups tends to surprise even the founders.

small variations in growth rate produce qualitatively different outcomes. that's why there's a separate word for startups, and why startups do things that ordinary companies don't, like raising money and getting acquired. and, strangely enough, it's also why they fail so frequently.

considering how valuable a successful startup can become, anyone familiar with the concept of expected value would be surprised if the failure rate weren't high. if a successful startup could make a founder $100 million, then even if the chance of succeeding were only 1%, the expected value of starting one would be $1 million. and the probability of a group of sufficiently smart and determined founders succeeding on that scale might be significantly over 1%. for the right people — e.g. the young bill gates — the probability might be 20% or even 50%. so it's not surprising that so many want to take a shot at it. in an efficient market, the number of failed startups should be proportionate to the size of the successes. and since the latter is huge the former should be too. [11]

what this means is that at any given time, the great majority of startups will be working on something that's never going to go anywhere, and yet glorifying their doomed efforts with the grandiose title of "startup."

this doesn't bother me. it's the same with other high-beta vocations, like being an actor or a novelist. i've long since gotten used to it. but it seems to bother a lot of people, particularly those who've started ordinary businesses. many are annoyed that these so-called startups get all the attention, when hardly any of them will amount to anything.

if they stepped back and looked at the whole picture they might be less indignant. the mistake they're making is that by basing their opinions on anecdotal evidence they're implicitly judging by the median rather than the average. if you judge by the median startup, the whole concept of a startup seems like a fraud. you have to invent a bubble to explain why founders want to start them or investors want to fund them. but it's a mistake to use the median in a domain with so much variation. if you look at the average outcome rather than the median, you can understand why investors like them, and why, if they aren't median people, it's a rational choice for founders to start them.

deals

why do investors like startups so much? why are they so hot to invest in photo-sharing apps, rather than solid money-making businesses? not only for the obvious reason.

the combination of founders, investors, and acquirers forms a natural ecosystem. it works so well that those who don't understand it are driven to invent conspiracy theories to explain how neatly things sometimes turn out. just as our ancestors did to explain the apparently too neat workings of the natural world. but there is no secret cabal making it all work.

if you start from the mistaken assumption that instagram was worthless, you have to invent a secret boss to force mark zuckerberg to buy it. to anyone who knows mark zuckerberg, that is the reductio ad absurdum of the initial assumption. the reason he bought instagram was that it was valuable and dangerous, and what made it so was growth.

if you want to understand startups, understand growth. growth drives everything in this world. growth is why startups usually work on technology — because ideas for fast growing companies are so rare that the best way to find new ones is to discover those recently made viable by change, and technology is the best source of rapid change. growth is why it's a rational choice economically for so many founders to try starting a startup: growth makes the successful companies so valuable that the expected value is high even though the risk is too. growth is why vcs want to invest in startups: not just because the returns are high but also because generating returns from capital gains is easier to manage than generating returns from dividends. growth explains why the most successful startups take vc money even if they don't need to: it lets them choose their growth rate. and growth explains why successful startups almost invariably get acquisition offers. to acquirers a fast-growing company is not merely valuable but dangerous too.

it's not just that if you want to succeed in some domain, you have to understand the forces driving it. understanding growth is what starting a startup *consists* of. what you're really doing (and to the dismay of some observers, all you're really doing) when you start a startup is committing to solve a harder type of problem than ordinary businesses do. you're committing to search for one of the rare ideas that generates rapid growth. because these ideas are so valuable, finding one is hard. the startup is the embodiment of your discoveries so far. starting a startup is thus very much like deciding to be a research scientist: you're not committing to solve any specific problem; you don't know for sure which problems are soluble; but you're committing to try to discover something no one knew before. a startup founder is in effect an economic research scientist. most don't discover anything that remarkable, but some discover relativity.

---

notes

1 strictly speaking it's not lots of customers you need but a big market, meaning a high product of number of customers times how much they'll pay. but it's dangerous to have too few customers even if they pay a lot, or the power that individual customers have over you could turn you into a de facto consulting firm. so whatever market you're in, you'll usually do best to err on the side of making the broadest type of product for it.

2 one year at startup school david heinemeier hansson encouraged programmers who wanted to start businesses to use a restaurant as a model. what he meant, i believe, is that it's fine to start software companies constrained in (a) in the same way a restaurant is constrained in (b). i agree. most people should not try to start startups.

3 that sort of stepping back is one of the things we focus on at y combinator. it's common for founders to have discovered something intuitively without understanding all its implications. that's probably true of the biggest discoveries in any field.

> may be one of the most valuable aspects of the rule of law. and so to the extent we see incumbents suppressing competitors via regulations or patent suits, we should worry, not because it's a departure from the rule of law per se but from what the rule of law is aiming at.

thanks to sam altman, marc andreessen, paul buchheit, patrick collison, jessica livingston, geoff ralston, and harj taggar for reading drafts of this.

october 2014

(this essay is derived from a guest lecture in sam altman's startup class at stanford. it's intended for college students, but much of it is applicable to potential founders at other ages.)

one of the advantages of having kids is that when you have to give advice, you can ask yourself "what would i tell my own kids?" my kids are little, but i can imagine what i'd tell them about startups if they were in college, and that's what i'm going to tell you.

startups are very counterintuitive. i'm not sure why. maybe it's just because knowledge about them hasn't permeated our culture yet. but whatever the reason, starting a startup is a task where you can't always trust your instincts.

it's like skiing in that way. when you first try skiing and you want to slow down, your instinct is to lean back. but if you lean back on skis you fly down the hill out of control. so part of learning to ski is learning to suppress that impulse. eventually you get new habits, but at first it takes a conscious effort. at first there's a list of things you're trying to remember as you start down the hill.

startups are as unnatural as skiing, so there's a similar list for startups. here i'm going to give you the first part of it — the things to remember if you want to prepare yourself to start a startup.

counterintuitive

the first item on it is the fact i already mentioned: that startups are so weird that if you trust your instincts, you'll make a lot of mistakes. if you know nothing more than this, you may at least pause before making them.

when i was running y combinator i used to joke that our function was to tell founders things they would ignore. it's really true. batch after batch, the yc partners warn founders about mistakes they're about to make, and the founders ignore them, and then come back a year later and say "i wish we'd listened."

why do the founders ignore the partners' advice? well, that's the thing about counterintuitive ideas: they contradict your intuitions. they seem wrong. so of course your first impulse is to disregard them. and in fact my joking description is not merely the curse of y combinator but part of its raison d'etre. if founders' instincts already gave them the right answers, they wouldn't need us. you only need other people to give you advice that surprises you. that's why there are a lot of ski instructors and not many running instructors. [1]

you can, however, trust your instincts about people. and in fact one of the most common mistakes young founders make is not to do that enough. they get involved with people who seem impressive, but about whom they feel some misgivings personally. later when things blow up they say "i knew there was something off about him, but i ignored it because he seemed so impressive."

if you're thinking about getting involved with someone — as a cofounder, an employee, an investor, or an acquirer — and you have misgivings about them, trust your gut. if someone seems slippery, or bogus, or a jerk, don't ignore it.

this is one case where it pays to be self-indulgent. work with people you genuinely like, and you've known long enough to be sure.

expertise

so this is the third counterintuitive thing to remember about startups: starting a startup is where gaming the system stops working. gaming the system may continue to work if you go to work for a big company. depending on how broken the company is, you can succeed by sucking up to the right people, giving the impression of productivity, and so on. [2] but that doesn't work with startups. there is no boss to trick, only users, and all users care about is whether your product does what they want. startups are as impersonal as physics. you have to make something people want, and you prosper only to the extent you do.

the dangerous thing is, faking does work to some degree on investors. if you're super good at sounding like you know what you're talking about, you can fool investors for at least one and perhaps even two rounds of funding. but it's not in your interest to. the company is ultimately doomed. all you're doing is wasting your own time riding it down.

so stop looking for the trick. there are tricks in startups, as there are in any domain, but they are an order of magnitude less important than solving the real problem. a founder who knows nothing about fundraising but has made something users love will have an easier time raising money than one who knows every trick in the book but has a flat usage graph. and more importantly, the founder who has made something users love is the one who will go on to succeed after raising the money.

though in a sense it's bad news in that you're deprived of one of your most powerful weapons, i think it's exciting that gaming the system stops working when you start a startup. it's exciting that there even exist parts of the world where you win by doing good work. imagine how depressing the world would be if it were all like school and big companies, where you either have to spend a lot of time on bullshit things or lose to people who do. [3] i would have been delighted if i'd realized in college that there were parts of the real world where gaming the system mattered less than others, and a few where it hardly mattered at all. but there are, and this variation is one of the most important things to consider when you're thinking about your future. how do you win in each type of work, and what would you like to win by doing? [4]

all-consuming

that brings us to our fourth counterintuitive point: startups are all-consuming. if you start a startup, it will take over your life to a degree you cannot imagine. and if your startup succeeds, it will take over your life for a long time: for several years at the very least, maybe for a decade, maybe for the rest of your working life. so there is a real opportunity cost here.

larry page may seem to have an enviable life, but there are aspects of it that are unenviable. basically at 25 he started running as fast as he could and it must seem to him that he hasn't stopped to catch his breath since. every day new shit happens in the google empire that only the ceo can deal with, and he, as ceo, has to deal with it. if he goes on vacation for even a week, a whole week's backlog of shit accumulates. and he has to bear this uncomplainingly, partly because as the company's daddy he can never show fear or weakness, and partly because billionaires get less than zero sympathy if they talk about having difficult lives. which has the strange side effect that the difficulty of being a successful startup founder is concealed from almost everyone except those who've done it.

y combinator has now funded several companies that can be called big successes, and in every single case the founders say the same thing. it never gets any easier. the nature of the problems change. you're worrying about construction delays at your london office instead of the broken air conditioner in your studio apartment. but the total volume of worry never decreases; if anything it increases.

should you do it at any age? i realize i've made startups sound pretty hard. if i haven't, let me try again: starting a startup is really hard. what if it's too hard? how can you tell if you're up to this challenge?

the answer is the fifth counterintuitive point: you can't tell. your life so far may have given you some idea what your prospects might be if you tried to become a mathematician, or a professional football player. but unless you've had a very strange life you haven't done much that was like being a startup founder. starting a startup will change you a lot. so what you're trying to estimate is not just what you are, but what you could grow into, and who can do that?

for the past 9 years it was my job to predict whether people would have what it took to start successful startups. it was easy to tell how smart they were, and most people reading this will be over that threshold. the hard part was predicting how tough and ambitious they would become. there may be no one who has more experience at trying to predict that, so i can tell you how much an expert can know about it, and the answer is: not much. i learned to keep a completely open mind about which of the startups in each batch would turn out to be the stars.

the founders sometimes think they know. some arrive feeling sure they will ace y combinator just as they've aced every one of the (few, artificial, easy) tests they've faced in life so far. others arrive wondering how they got in, and hoping yc doesn't discover whatever mistake caused it to accept them. but there is little correlation between founders' initial attitudes and how well their companies do.

i've read that the same is true in the military — that the swaggering recruits are no more likely to turn out to be really tough than the quiet ones. and probably for the same reason: that the tests involved are so different from the ones in their previous lives.

if you're absolutely terrified of starting a startup, you probably shouldn't do it. but if you're merely unsure whether you're up to it, the only way to find out is to try. just not now.

ideas

so if you want to start a startup one day, what should you do in college? there are only two things you need initially: an idea and cofounders. and the m.o. for getting both is the same. which leads to our sixth and last counterintuitive point: that the way to get startup ideas is not to try to think of startup ideas.

i've written a whole essay on this, so i won't repeat it all here. but the short version is that if you make a conscious effort to think of startup ideas, the ideas you come up with will not merely be bad, but bad and plausible-sounding, meaning you'll waste a lot of time on them before realizing they're bad.

the way to come up with good startup ideas is to take a step back. instead of making a conscious effort to think of startup ideas, turn your mind into the type that startup ideas form in without any conscious effort. in fact, so unconsciously that you don't even realize at first that they're startup ideas.

this is not only possible, it's how apple, yahoo, google, and facebook all got started. none of these companies were even meant to be companies at first. they were all just side projects. the best startups almost have to start as side projects, because great ideas tend to be such outliers that your conscious mind would reject them as ideas for companies.

ok, so how do you turn your mind into the type that startup ideas form in unconsciously? (1) learn a lot about things that matter, then (2) work on problems that interest you (3) with people you like and respect. the third part, incidentally, is how you get cofounders at the same time as the idea.

by genuine curiosity, not some ulterior motive.

at its best, starting a startup is merely an ulterior motive for curiosity. and you'll do it best if you introduce the ulterior motive toward the end of the process.

so here is the ultimate advice for young would-be startup founders, boiled down to two words: just learn.

---

notes

1 some founders listen more than others, and this tends to be a predictor of success. one of the things i remember about the airbnbs during yc is how intently they listened.

2 in fact, this is one of the reasons startups are possible. if big companies weren't plagued by internal inefficiencies, they'd be proportionately more effective, leaving less room for startups.

3 in a startup you have to spend a lot of time on schleps, but this sort of work is merely unglamorous, not bogus.

4 what should you do if your true calling is gaming the system? management consulting.

5 the company may not be incorporated, but if you start to get significant numbers of users, you've started it, whether you realize it yet or not.

6 it shouldn't be that surprising that colleges can't teach students how to be good startup founders, because they can't teach them how to be good employees either.

the way universities "teach" students how to be employees is to hand off the task to companies via internship programs. but you couldn't do the equivalent thing for startups, because by definition if the students did well they would never come back.

7 charles darwin was 22 when he received an invitation to travel aboard the hms beagle as a naturalist. it was only because he was otherwise unoccupied, to a degree that alarmed his family, that he could accept it. and yet if he hadn't we probably would not know his name.

8 parents can sometimes be especially conservative in this department. there are some whose definition of important problems includes only those on the critical path to med school.

9 i did manage to think of a heuristic for detecting whether you have a taste for interesting ideas: whether you find known boring ideas intolerable. could you endure studying literary theory, or working in middle management at a large company?

10 in fact, if your goal is to start a startup, you can stick even more closely to the ideal of a liberal education than past generations have. back when students focused mainly on getting a job after college, they thought at least a little about how the courses they took might look to an employer. and perhaps even worse, they might shy away from taking a difficult class lest they get a low grade, which would harm their all-important gpa. good news: users don't care what your gpa was. and i've never heard of investors caring either. y combinator certainly never asks what classes you took in college or what grades you got in them.

thanks to sam altman, paul buchheit, john collison, patrick collison, jessica livingston, robert morris, geoff ralston, and fred wilson for reading drafts of this.

watch how this essay was written.

february 2009

one of the things i always tell startups is a principle i learned from paul buchheit: it's better to make a few people really happy than to make a lot of people semi-happy. i was saying recently to a reporter that if i could only tell startups 10 things, this would be one of them. then i thought: what would the other 9 be?

when i made the list there turned out to be 13:

1. pick good cofounders.

cofounders are for a startup what location is for real estate. you can change anything about a house except where it is. in a startup you can change your idea easily, but changing your cofounders is hard. [1] and the success of a startup is almost always a function of its founders.

2. launch fast.

the reason to launch fast is not so much that it's critical to get your product to market early, but that you haven't really started working on it till you've launched. launching teaches you what you should have been building. till you know that you're wasting your time. so the main value of whatever you launch with is as a pretext for engaging users.

3. let your idea evolve.

this is the second half of launching fast. launch fast and iterate. it's a big mistake to treat a startup as if it were merely a matter of implementing some brilliant initial idea. as in an essay, most of the ideas appear in the implementing.

4. understand your users.

you can envision the wealth created by a startup as a rectangle, where one side is the number of users and the other is how much you improve their lives. [2] the second dimension is the one you have most control over. and indeed, the growth in the first will be driven by how well you do in the second. as in science, the hard part is not answering questions but asking them: the hard part is seeing something new that users lack. the better you understand them the better the odds of doing that. that's why so many successful startups make something the founders needed.

5. better to make a few users love you than a lot ambivalent.

ideally you want to make large numbers of users love you, but you can't expect to hit that right away. initially you have to choose between satisfying all the needs of a subset of potential users, or satisfying a subset of the needs of all potential users. take the first. it's easier to expand userwise than satisfactionwise. and perhaps more importantly, it's harder to lie to yourself. if you think you're 85% of the way to a great product, how do you know it's not 70%? or 10%? whereas it's easy to know how many users you have.

6. offer surprisingly good customer service.

customers are used to being maltreated. most of the companies they deal with are quasi-monopolies that get away with atrocious customer service. your own ideas about what's possible have been uncons-

processes that we should ignore till they terminated. it's very dangerous to morale to start to depend on deals closing, not just because they so often don't, but because it makes them less likely to.

having gotten it down to 13 sentences, i asked myself which i'd choose if i could only keep one.

understand your users. that's the key. the essential task in a startup is to create wealth; the dimension of wealth you have most control over is how much you improve users' lives; and the hardest part of that is knowing what to make for them. once you know what to make, it's mere effort to make it, and most decent hackers are capable of that.

understanding your users is part of half the principles in this list. that's the reason to launch early, to understand your users. evolving your idea is the embodiment of understanding your users. understanding your users well will tend to push you toward making something that makes a few people deeply happy. the most important reason for having surprisingly good customer service is that it helps you understand your users. and understanding your users will even ensure your morale, because when everything else is collapsing around you, having just ten users who love you will keep you going.

---

notes

1 strictly speaking it's impossible without a time machine.

2 in practice it's more like a ragged comb.

3 joe thinks one of the founders of hewlett packard said it first, but he doesn't remember which.

4 they'd be interchangeable if markets stood still. since they don't, working twice as fast is better than having twice as much time.

i met the reddits before we even started y combinator. in fact they were one of the reasons we started it.

yc grew out of a talk i gave to the harvard computer society (the undergrad computer club) about how to start a startup. everyone else in the audience was probably local, but steve and alexis came up on the train from the university of virginia, where they were seniors. since they'd come so far i agreed to meet them for coffee. they told me about the startup idea we'd later fund them to drop: a way to order fast food on your cellphone.

this was before smartphones. they'd have had to make deals with cell carriers and fast food chains just to get it launched. so it was not going to happen. it still doesn't exist, 19 years later. but i was impressed with their brains and their energy. in fact i was so impressed with them and some of the other people i met at that talk that i decided to start something to fund them. a few days later i told steve and alexis that we were starting y combinator, and encouraged them to apply.

that first batch we didn't have any way to identify applicants, so we made up nicknames for them. the reddits were the "cell food muffins." "muffin" is a term of endearment jessica uses for things like small dogs and two year olds. so that gives you some idea what kind of impression steve and alexis made in those days. they had the look of slightly ruffled surprise that baby birds have.

their idea was bad though. and since we thought then that we were funding ideas rather than founders, we rejected them. but we felt bad about it. jessica was sad that we'd rejected the muffins. and it seemed wrong to me to turn down the people we'd been inspired to start yc to fund.

i don't think the startup sense of the word "pivot" had been invented yet, but we wanted to fund steve and alexis, so if their idea was bad, they'd have to work on something else. and i knew what else. in those days there was a site called delicious where you could save links. it had a page called del.icio.us/popular that listed the most-saved links, and people were using this page as a de facto reddit. i knew because a lot of the traffic to my site was coming from it. there needed to be something like del.icio.us/popular, but designed for sharing links instead of being a byproduct of saving them.

so i called steve and alexis and said that we liked them, just not their idea, so we'd fund them if they'd work on something else. they were on the train home to virginia at that point. they got off at the next station and got on the next train north, and by the end of the day were committed to working on what's now called reddit.

they would have liked to call it snoo, as in "what snoo?" but snoo.com was too expensive, so they settled for calling the mascot snoo and picked a name for the site that wasn't registered. early on reddit was just a provisional name, or so they told me at least, but it's probably too late to change it now.

as with all the really great startups, there's an uncannily close match between the company and the founders. steve in particular. reddit has a certain personality — curious, skeptical, ready to be amused — and that personality is steve's.

steve will roll his eyes at this, but he's an intellectual; he's interested in ideas for their own sake. that was how he came to be in that audience in cambridge in the first place. he knew me because he was interested in a programming language i've written about called lisp, and lisp is one of those languages few people learn except out of intellectual curiosity. steve's kind of vacuum-cleaner curiosity is exactly what you want when you're starting a site that's a list of links to literally anything interesting.

steve was not a big fan of authority, so he also liked the idea of a site without editors. in those days the top forum for programmers was a site called slashdot. it was a lot like reddit, except the stories on the

(this is a talk i gave to 14 and 15 year olds about what to do now if they might want to start a startup later. lots of schools think they should tell students something about startups. this is what i think they should tell them.)

most of you probably think that when you're released into the so-called real world you'll eventually have to get some kind of job. that's not true, and today i'm going to talk about a trick you can use to avoid ever having to get a job.

the trick is to start your own company. so it's not a trick for avoiding *work*, because if you start your own company you'll work harder than you would if you had an ordinary job. but you will avoid many of the annoying things that come with a job, including a boss telling you what to do.

it's more exciting to work on your own project than someone else's. and you can also get a lot richer. in fact, this is the standard way to get really rich. if you look at the lists of the richest people that occasionally get published in the press, nearly all of them did it by starting their own companies.

starting your own company can mean anything from starting a barber shop to starting google. i'm here to talk about one extreme end of that continuum. i'm going to tell you how to start google.

the companies at the google end of the continuum are called startups when they're young. the reason i know about them is that my wife jessica and i started something called y combinator that is basically a startup factory. since 2005, y combinator has funded over 4000 startups. so we know exactly what you need to start a startup, because we've helped people do it for the last 19 years.

you might have thought i was joking when i said i was going to tell you how to start google. you might be thinking "how could *we* start google?" but that's effectively what the people who did start google were thinking before they started it. if you'd told larry page and sergey brin, the founders of google, that the company they were about to start would one day be worth over a trillion dollars, their heads would have exploded.

all you can know when you start working on a startup is that it seems worth pursuing. you can't know whether it will turn into a company worth billions or one that goes out of business. so when i say i'm going to tell you how to start google, i mean i'm going to tell you how to get to the point where you can start a company that has as much chance of being google as google had of being google. [1]

how do you get from where you are now to the point where you can start a successful startup? you need three things. you need to be good at some kind of technology, you need an idea for what you're going to build, and you need cofounders to start the company with.

how do you get good at technology? and how do you choose which technology to get good at? both of those questions turn out to have the same answer: work on your own projects. don't try to guess whether gene editing or llms or rockets will turn out to be the most valuable technology to know about. no one can predict that. just work on whatever interests you the most. you'll work much harder on something you're interested in than something you're doing because you think you're supposed to.

if you're not sure what technology to get good at, get good at programming. that has been the source of the median startup for the last 30 years, and this is probably not going to change in the next 10.

those of you who are taking computer science classes in school may at this point be thinking, ok, we've got this sorted. we're already being taught all about programming. but sorry, this is not enough. you have to be working on your own projects, not just learning stuff in classes. you can do well in computer science classes without ever really learning to program. in fact you can graduate with a degree in computer science from a top university and still not be any good at programming. that's why tech compan-

tell you what these directories were called, you'll know which startup i'm talking about. they were called facebooks, because they usually had a picture of each student next to their name.

so mark zuckerberg shows up at harvard in 2002, and the university still hasn't gotten the facebook online. each individual house has an online facebook, but there isn't one for the whole university. the university administration has been diligently having meetings about this, and will probably have solved the problem in another decade or so. most of the students don't consciously notice that anything is wrong. but mark is a programmer. he looks at this situation and thinks "well, this is stupid. i could write a program to fix this in one night. just let people upload their own photos and then combine the data into a new site for the whole university." so he does. and almost literally overnight he has thousands of users.

of course facebook was not a startup yet. it was just a... project. there's that word again. projects aren't just the best way to learn about technology. they're also the best source of startup ideas.

facebook was not unusual in this respect. apple and google also began as projects. apple wasn't meant to be a company. steve wozniak just wanted to build his own computer. it only turned into a company when steve jobs said "hey, i wonder if we could sell plans for this computer to other people." that's how apple started. they weren't even selling computers, just plans for computers. can you imagine how lame this company seemed?

ditto for google. larry and sergey weren't trying to start a company at first. they were just trying to make search better. before google, most search engines didn't try to sort the results they gave you in order of importance. if you searched for "rugby" they just gave you every web page that contained the word "rugby." and the web was so small in 1997 that this actually worked! kind of. there might only be 20 or 30 pages with the word "rugby," but the web was growing exponentially, which meant this way of doing search was becoming exponentially more broken. most users just thought, "wow, i sure have to look through a lot of search results to find what i want." door sticks. but like mark, larry and sergey were programmers. like mark, they looked at this situation and thought "well, this is stupid. some pages about rugby matter more than others. let's figure out which those are and show them first."

it's obvious in retrospect that this was a great idea for a startup. it wasn't obvious at the time. it's never obvious. if it was obviously a good idea to start apple or google or facebook, someone else would have already done it. that's why the best startups grow out of projects that aren't meant to be startups. you're not trying to start a company. you're just following your instincts about what's interesting. and if you're young and good at technology, then your unconscious instincts about what's interesting are better than your conscious ideas about what would be a good company.

so it's critical, if you're a young founder, to build things for yourself and your friends to use. the biggest mistake young founders make is to build something for some mysterious group of other people. but if you can make something that you and your friends truly want to use — something your friends aren't just using out of loyalty to you, but would be really sad to lose if you shut it down — then you almost certainly have the germ of a good startup idea. it may not seem like a startup to you. it may not be obvious how to make money from it. but trust me, there's a way.

what you need in a startup idea, and all you need, is something your friends actually want. and those ideas aren't hard to see once you're good at technology. there are sticking doors everywhere. [2]

now for the third and final thing you need: a cofounder, or cofounders. the optimal startup has two or three founders, so you need one or two cofounders. how do you find them? can you predict what i'm going to say next? it's the same thing: projects. you find cofounders by working on projects with them.

at a yc event last week brian chesky gave a talk that everyone who was there will remember. most founders i talked to afterward said it was the best they'd ever heard. ron conway, for the first time in his life, forgot to take notes. i'm not going to try to reproduce it here. instead i want to talk about a question it raised.

the theme of brian's talk was that the conventional wisdom about how to run larger companies is mistaken. as airbnb grew, well-meaning people advised him that he had to run the company in a certain way for it to scale. their advice could be optimistically summarized as "hire good people and give them room to do their jobs." he followed this advice and the results were disastrous. so he had to figure out a better way on his own, which he did partly by studying how steve jobs ran apple. so far it seems to be working. airbnb's free cash flow margin is now among the best in silicon valley.

the audience at this event included a lot of the most successful founders we've funded, and one after another said that the same thing had happened to them. they'd been given the same advice about how to run their companies as they grew, but instead of helping their companies, it had damaged them.

why was everyone telling these founders the wrong thing? that was the big mystery to me. and after mulling it over for a bit i figured out the answer: what they were being told was how to run a company you hadn't founded — how to run a company if you're merely a professional manager. but this m.o. is so much less effective that to founders it feels broken. there are things founders can do that managers can't, and not doing them feels wrong to founders, because it is.

in effect there are two different ways to run a company: founder mode and manager mode. till now most people even in silicon valley have implicitly assumed that scaling a startup meant switching to manager mode. but we can infer the existence of another mode from the dismay of founders who've tried it, and the success of their attempts to escape from it.

there are as far as i know no books specifically about founder mode. business schools don't know it exists. all we have so far are the experiments of individual founders who've been figuring it out for themselves. but now that we know what we're looking for, we can search for it. i hope in a few years founder mode will be as well understood as manager mode. we can already guess at some of the ways it will differ.

the way managers are taught to run companies seems to be like modular design in the sense that you treat subtrees of the org chart as black boxes. you tell your direct reports what to do, and it's up to them to figure out how. but you don't get involved in the details of what they do. that would be micromanaging them, which is bad.

hire good people and give them room to do their jobs. sounds great when it's described that way, doesn't it? except in practice, judging from the report of founder after founder, what this often turns out to mean is: hire professional fakers and let them drive the company into the ground.

one theme i noticed both in brian's talk and when talking to founders afterward was the idea of being gaslit. founders feel like they're being gaslit from both sides — by the people telling them they have to run their companies like managers, and by the people working for them when they do. usually when everyone around you disagrees with you, your default assumption should be that you're mistaken. but this is one of the rare exceptions. vcs who haven't been founders themselves don't know how founders should run companies, and c-level execs, as a class, include some of the most skillful liars in the world. [1]

whatever founder mode consists of, it's pretty clear that it's going to break the principle that the ceo should engage with the company only via his or her direct reports. "skip-level" meetings will become

how should universities prepare students to start startups? y combinator is in the perfect position to answer this question, because we get them next. we're like grad school. and because yc has had 20 years to refine its model of what a promising founder looks like, you probably won't find a better target.

what do the yc partners look for? it's surprisingly simple. they want people who are good at building things and have a habit of doing it.

the hard part of startups is product: knowing what to build, and being able to build it. and that kind of knowledge comes from studying computer science or mechanical engineering or molecular biology, not management or finance. [1]

so the way to prepare undergraduates to become successful founders is not to give them some new curriculum focused on "entrepreneurship". it's to do what universities already do best — to teach them computer science and mechanical engineering and molecular biology. [2]

indeed, preparing students to start startups is closer to the ideal of liberal education than preparing them for almost any other kind of career. startups succeed or fail based on how much customers like the product. customers don't care what the founders studied in college. so founders are free to study whatever they want, as long as they get good at building things.

but building should be understood in a very broad sense. it doesn't mean all would-be founders have to study some form of engineering. almost any kind of expertise that could be described as building or creating could be useful. it was useful to steve jobs to have studied calligraphy, for example; it was one of the reasons apple dominated desktop publishing. so while math and science and engineering and design tend to be good bets, i would not want to draw a sharp line around them, because i can imagine other forms of building that could be useful. and of course you don't have to major in something to be good at it. mark zuckerberg was good at programming, but he was a psychology major, not a cs major.

the best way to describe what would-be founders should study is that they should seek out powerful ideas. but smart people are naturally attracted to powerful ideas anyway. so as long as departments teaching powerful ideas exist, the sort of people who'd make good founders will find them. [3]

in fact there are only two things universities need to change to be perfect at preparing founders: they need to make students feel that starting a startup is something they can do, and they need to encourage them to work on their own projects.

at the moment, the belief that it's possible to start a startup is very unevenly distributed. yc now gets so many applications that our application data is a reasonable proxy for interest in startups at different universities, and harvard alumni, for example, apply at about twice the rate of yale and princeton alumni. presumably harvard students aren't that different from yale and princeton students; the reason harvard students go on to start more startups is just that it's more customary there. which in turn implies that merely by making their students feel that starting a startup is a viable option, yale and princeton could at least double the number who do.

once a university has a culture of starting startups, you don't have to convince students that it's a viable option. new students learn that from older ones. but at a university that doesn't have much of a startup culture yet, there are things you can do to help this realization along. the most effective is probably to show students examples of people who've done it.

until you've seen some founders in real life, you tend to think that starting startups is something done by other people. seeing them pops that bubble. in fact seeing founders in real life is doubly inspiring:

few weeks resulted in two trillion dollar companies. imagine what the us gdp would be if reading period at harvard were twice as long.

universities will tend to resist the idea of keeping students less busy with coursework. partly because administrators feel that if they want to achieve something, they have to do it by taking active measures. achieving something merely by leaving students alone is alien to their nature.

and they should be left alone. these things should be the students' own projects; the university should resist the temptation to make them official. partly because students will be more excited to work on a project that's entirely their own, and partly because many projects wouldn't survive official recognition, because they break some sort of rule. bill gates and mark zuckerberg both got in trouble with the harvard administration over projects they worked on as undergrads. bill broke university rules by bringing paul allen, who wasn't a student, into the computer lab with him to work on altair basic. zuck got in such trouble over facemash that he was put on disciplinary probation. and their cases are probably more the rule than the exception. universities have lots of rules, and novel projects are often untidy things.

right now there are students flying drones out of line of sight. turn a blind eye to it.

another reason it will be hard for universities to keep students less busy is that they'll worry that without some kind of oversight, most students will just waste whatever free time they're given. and they will! the price of giving the most energetic students room to do even better is that it leaves the least energetic ones room to do even worse. but that's a price that's worth paying, because if the most energetic students do better they could do a *lot* better, whereas the laziest students already learn so little that there's not much room for them to do worse. so giving all the students some of their time back could improve the average outcome a lot, even if it doesn't move the median. [4]

it may seem a bit excessive to change the whole schedule of the university just to encourage would-be founders. they're never going to be more than 10% of the students. and it probably would be excessive if this change only helped founders. but in fact giving the students some of their time back would help all the most energetic and ambitious ones. they'd all explore new things of one type or another if the pressure of work were relieved for even a week or two.

---

now that i've explained how universities should prepare founders, i should explain how not to. one thing universities can't do is actually teach students how to start startups. starting a startup is one of those things, like chemistry or painting, that you have to learn by doing. which means a properly run class on how to start a startup would have to be a lab class: the students would actually have to start startups. and i know exactly what a class of this type should look like, because yc is it. but yc is very different in structure from a university, and if you tried to cram it into an undergrad degree program, it would become a joke. are the students supposed to start these companies without any funding? are they supposed to run startups, which notoriously take every moment of your time when done properly, while simultaneously taking three or four other classes? and what if, despite these handicaps, some of the startups actually take off? are the students just supposed to abandon them? because it's either that or drop out.

running a startup is incompatible with being a full time student. the only way to learn how to start a startup is to do it. those two statements are so obvious that they're practically truisms. and yet so many people manage to remain in denial about what they imply. you can't teach students how to start startups.

4 there's an interesting parallel here to variation in income. the bottom of the income scale is anchored firmly at zero, because there are some people who are either incapable of working or just not interested in doing it at the moment. if you let there be more variation in income, it won't affect the income of the people at this end of the scale; n times zero is zero; but at the other end of the scale you'll see enormous change.

5 presumably one reason these competitions lean toward impressing investors rather than users is that it's the only way to have a single set of judges. investors can be treated as interchangeable, whereas the users of each product might be different. but if it's impractical to measure the right thing, that doesn't mean the solution is to measure the wrong one.

6 another thing that will tend to draw universities away from the optimal path is business schools, if they have them. business schools were not designed to train founders. they were designed to train the managerial class of the large industrial companies that arose in the early 20th century; they're the west points of industrial capitalism. that's why their official name is usually the school of management. but while the skills they teach might be useful in running companies beyond a certain size, they're not the critical ingredient in *founding* them. and the skills that are are already taught by other departments. so to the extent business schools affect their parent university's strategy for preparing founders, it can only be by adding error.

thanks to trevor blackwell, daniel diermeier, jared friedman, diana hu, michael kotlikoff, jessica livingston, robert morris, harj taggar, and garry tan for reading drafts of this.

(this is based on a talk i gave at the oxford union.)

since this is apparently the future prime ministers' club, i'm going to tell you about something it would be good if more politicians understood: i'm going to tell you how people become billionaires. i hope this will be useful to you even if you don't plan to go into politics. those of you who don't become prime minister can become billionaires instead.

the reason i know about this topic is that 21 years ago jessica and i started something called y combinator. if you haven't heard of y combinator, it's a cross between an investment firm and a school for startup founders. since we started it in 2005 we've funded about 6500 companies.

starting a successful startup is the most common way to become a billionaire, so in effect i've spent the last 21 years training people to become billionaires. so far about 30 of them have, but there are many more in the pipeline.

so you can imagine how astonished i was last month when an american politician said that it was impossible to earn a billion dollars. i felt like a skating coach hearing someone say that it's impossible to do a triple axel. of course it's possible. it's *hard*, but it's possible.

she wasn't saying, of course, that it's impossible to become a billionaire. obviously that's possible. nor was she talking about the distinction between income and capital gains. she wasn't making a point about accounting. what she meant was that it's impossible to get that rich without doing something bad — without cheating in some way.

a couple days later i was talking to the founder of a startup i'd funded. i began by asking, as i usually do when i meet a founder, what her growth rate was. 93% last month, she said. i pointed out that this meant her net worth was also growing at 93% a month. she was getting richer at a stupendously rapid rate. and yet she hadn't been doing anything bad. the reason her startup was growing so fast was simply that users loved what she'd built. so she could feel from her own experience how wrong that politician was. she wasn't exploiting anyone. exactly the opposite in fact. the reason her startup was growing so fast was that she and her cofounder had been working their asses off to make their users happy, and as a result the users had been telling their friends. and that gets you exponential growth.

later that day i was talking about her case online, and someone replied that having a few million and growing at 93% a month was radically different from being a billionaire.

i suspect many people would agree with this statement. but it turns out not merely to be false, but false in a very illuminating way.

so i would like you all to do me a favor please. i would like you to take out your phones and calculate a number. i know this may seem contrived, but i promise it will be useful for you. i'm going to have you do the most common kind of calculation i do as an investor, and the experience will bring home to you what startups are all about.

if we interpret his statement in the most conservative way and assume that a few means 2, her company has to grow 500x for her to become a billionaire. so we are going to calculate how many months of 93% growth it takes for something to grow 500x.

to do this we want to calculate the log base 1.93 of 500. the easiest way to do that is to go to google search, which lets you do calculations right in the search box. so go to google search and type log(500, 1.93). if you typed that right, the answer you get is about 9.45.

you're young, and usually young founders should make something that they themselves want. you don't have enough experience yet to know what other people need. but at the same time your own needs are uniquely valuable, because your needs predict future demand. you're the age when people start using new things. whatever you and your friends start using now, everyone is going to be using in ten years. since your intuitions about other people's needs are usually a crap signal, and your own needs are an especially valuable one, you should usually listen to the second signal; you should make something you and your friends want.

making something you and your friends want doesn't mean you have to build a consumer product. maybe you and your friends are molecular biologists, and there's something cool that could be done now to dna that everyone else has overlooked. maybe you and your friends are into drones. the idea doesn't have to have a wide appeal. it literally just has to appeal to you and your friends.

don't worry about the second number, the market size. since you predict future demand, the market will grow. and it's always possible to expand into adjacent markets. all you need is a beachhead in the territory of unsatisfied need that you can expand from.

how do you get an idea like that? the answer is one of the most counterintuitive things about startups — which is saying something, because there are a lot of counterintuitive things about startups. but the way to get the very best startup ideas is not to look for startup ideas. if you're consciously looking for startup ideas, it will make you too conservative. you'll lop off the outliers. because the very best startup ideas tend to sound so lame, at first, that you'd reject them if you were consciously looking for startup ideas. that's what has prevented them from being discovered.

imagine what a bad idea apple or facebook or airbnb seemed at first. how many people are going to want their own computers? how is a company going to make money from undergrads stalking one another online? who's going to pay to sleep on an airbed on someone's floor? we know how these ideas turned out, so it's easy to rewrite history, but i remember very well how bad facebook and airbnb sounded at first. we *funded* airbnb, and we thought the idea was bad. the reason we funded them was just that we liked the founders.

so how do you find startup ideas without looking for them? by working on projects with your friends. that's where the very best startups come from. initially they're not even meant to be companies. they're just something people built because they thought it would be cool. that's how apple and google and facebook all started. none of them were meant to be companies at first.

the reason this works is what i told you earlier: you predict future demand. so if you just build random stuff you think would be cool, the things you build will actually be far from random.

this is one of those cases where your unconscious mind knows more than your conscious mind does. anything that genuinely seems to you like it would be a cool thing to build has a high probability of leading to a good startup idea, no matter how preposterous it sounds. whatever you build couldn't possibly sound more preposterous than a startup we funded in 2006 called justin.tv. it consisted of one guy, justin kan, walking around with a camera on the side of his head, live streaming everything that happened to him. but this company ended up doing quite well. in fact you've probably heard of it, but under its new name, twitch.

the key to starting a successful startup is to understand some group of users so well that you can make exactly what they want. if you're young you can, and should, use the hack of making something for yourself. you understand yourself. but this is just an instance of the more general rule. only by unders-